NEXT GREATER ELEMENT

Using Pointers, Dynamic Arrays and Nested Loops

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

1. Problem Statement

Write a C program that reads the size and the elements of a variable-length integer array arr[]. The array must be passed to a function named next_greater() using its address, and pointers must be used to pass the array between the calling function and next_greater().

The function next_greater() must:

- Find the next greater element for each element in arr[].
- Search only the elements to the right of the current element and select the nearest element that is greater than the current element.
- Display -1 if no greater element exists to the right.
- Display the results in the order of appearance in the array, separated by commas.
- Use nested loops.

2. Objective

The objective of this project is to practise the core ideas of the first semester of C programming: reading input, dynamic memory allocation for a variable-length array, passing an array to a function through a pointer, pointer arithmetic, nested loops, and the break statement.

3. Concepts Used

- **Pointers:** the array is passed to next_greater() as an int * and elements are accessed with *(arr + i) instead of arr[i].
- **Dynamic memory allocation:** malloc() creates an array whose size is decided by the user at run time, and free() releases it.
- **Nested loops:** the outer loop selects each element, the inner loop scans the elements on its right.
- **Flag variable and break:** a variable found records whether a greater element was located; break stops the search at the nearest one.
- **Input validation:** the program rejects a size that is not a positive number and checks that malloc() succeeded.

4. Algorithm

1. Start.
2. Read the size n of the array. If n is not a positive integer, display an error message and stop.
3. Allocate memory for n integers using malloc() and read the n elements.
4. Call next_greater(arr, n), passing the base address of the array.
5. Inside next_greater(), repeat steps 6 to 9 for every index i from 0 to n - 1.
6. Set found = 0.
7. For every index j from i + 1 to n - 1: if *(arr + j) is greater than *(arr + i), print *(arr + j), set found = 1 and leave the inner loop with break.
8. If found is still 0, print -1.
9. If i is not the last index, print a comma and a space.
10. Print a new line, release the memory with free() and stop.

5. Source Code

File name: next_greater.c

```
/*
 * Program : Next Greater Element
 * Author  : Rudra Pratap Singh
 * SAP ID  : 590032811
 * Batch   : 16
 *
 * Description:
 * Reads the size and elements of a variable-length integer array and
 * prints the next greater element (to the right) for every element.
 * If no greater element exists on the right, -1 is printed.
 */

#include <stdio.h>
#include <stdlib.h>

/* Prints the next greater element of every element of arr[].
   The array is received through a pointer. */
void next_greater(int *arr, int n)
{
    int i, j, found;

    for (i = 0; i < n; i++)
    {
        found = 0;

        /* search only the elements to the right of arr[i] */
        for (j = i + 1; j < n; j++)
        {
            if (*(arr + j) > *(arr + i))
            {
```

```
                printf("%d", *(arr + j));   /* nearest greater element */
                found = 1;
                break;
            }
        }

        if (found == 0)
            printf("-1");                   /* no greater element on right */

        if (i < n - 1)
            printf(", ");                   /* separator between results */
    }
    printf("\n");
}

int main()
{
    int n, i;
    int *arr;

    printf("Enter the size of the array: ");
    if (scanf("%d", &n) != 1 || n <= 0)
    {
        printf("Invalid array size.\n");
        return 1;
    }

    /* allocate memory for a variable-length array */
    arr = (int *)malloc(n * sizeof(int));
    if (arr == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", arr + i);

    printf("Output:\n");
    next_greater(arr, n);               /* array passed using its address */

    free(arr);
    return 0;
}
```

6. Explanation of the Code

6.1 The main() function

main() reads the size n and checks that it is a positive integer. It then uses malloc() to create an array of n integers, which is how the variable-length array is obtained. The elements are read with scanf("%d", arr + i), where arr + i is the address of the i-th element. The base address arr is then passed to next_greater(), and the memory is released with free() at the end.

6.2 The next_greater() function

The function receives the pointer int *arr and the size n. The outer loop (variable i) visits every element. For each element, the inner loop (variable j) starts at i + 1, so only the elements to the right are examined. The first element found that is larger than *(arr + i) is the nearest greater element; it is printed and the inner loop ends with break. If the inner loop completes without finding one, found remains 0 and -1 is printed. A comma and a space are printed after every result except the last, so the output is correctly comma-separated and in the same order as the array.

6.3 Dry run for the array 4 5 2 25

i	arr[i]	Elements to right	Nearest greater
0	4	5, 2, 25	5
1	5	2, 25	25
2	2	25	25
3	25	(none)	-1

Final output: `5, 25, 25, -1`

7. Sample Input and Output

The program was compiled with gcc -Wall -Wextra without any warnings and executed on the sample cases given in the problem statement.

Example 1

```
Enter the size of the array: 4
Enter 4 elements: 4 5 2 25
Output:
5, 25, 25, -1
```

Example 2

```
Enter the size of the array: 4
Enter 4 elements: 13 7 6 12
Output:
-1, 12, 12, -1
```

Example 3

```
Enter the size of the array: 4
Enter 4 elements: 1 3 2 4
Output:
3, 4, 4, -1
```

Example 4

```
Enter the size of the array: 5
Enter 5 elements: 1 2 3 4 5
Output:
2, 3, 4, 5, -1
```

8. Test Summary

In addition to the four given examples, two boundary cases were tested: an array with a single element and a strictly descending array.

Test	Input Array	Output	Status
1	4 5 2 25	`5, 25, 25, -1`	**Passed**
2	13 7 6 12	`-1, 12, 12, -1`	**Passed**
3	1 3 2 4	`3, 4, 4, -1`	**Passed**
4	1 2 3 4 5	`2, 3, 4, 5, -1`	**Passed**
5	7 (single element)	`-1`	**Passed**
6	5 4 3 2 1 (descending)	`-1, -1, -1, -1, -1`	**Passed**

9. Complexity Analysis

- **Time complexity:** $O(n^2)$ in the worst case, because for each of the n elements the inner loop may scan up to n - 1 elements to its right.
- **Space complexity:** O(n) for the array itself; the function uses only a constant amount of extra memory (i, j and found).

10. Conclusion

The program correctly finds the next greater element for every element of a user-defined array. It satisfies all the given conditions: the array is created at run time, it is passed to next_greater() by its address, elements are accessed using pointers, nested loops perform the search, -1 is shown when no greater element exists to the right, and the results are displayed in order, separated by commas. All the sample cases produce exactly the expected output.